NAAN MUDHALVAN ARTS INTERNSHIP PROGRAM 2026

Frontend Web Development using React

Project Submission Template

STUDENT NAME:C.DHARSHINI

YEAR:III-B.SC-CS

COLLEGE CODE:brubz

COLLEGE NAME:GOVERNMENT ARTS AND SCIENCE,ANTHIYUR

DEPARTMENT:COMPUTER SCIENCE

SEMESTER:V

ROLL NO:2422k2705

NMID:asbrubz2422k2705

DATE:21.06.2026

TITLE:DAILY EXPENSE TRACKER ANAZLICSE

GITHUB LINK : https://github.com/dharshini200622/Expense_Tracker

VERCEL LINK: https://expense-tracker-theta-nine-97.vercel.app/

---

1. ABSTRACT

Expense Flow is a modern, responsive, client-side financial tracking dashboard developed using React.js and Vite to streamline personal asset monitoring.

Designed with a premium minimalist aesthetic inspired by platforms like Stripe and Linear, the application serves as an elegant, zero-overhead ledger that operates entirely in the browser.

By leveraging React's Context API for seamless state management and HTML5 Local Storage for persistent data caching, the system removes the requirement for complex backend integrations or remote database infrastructure.

Users can monitor real-time metrics including their overall balance, income flow, and systemic expense tracking with interactive structural charts powered by Chart.js.

2. PROBLEM STATEMENT

In an increasingly cashless economy, individuals frequently struggle to maintain granular visibility over daily micro-transactions, leading to unexpected budget deficits and poor financial planning.

Existing enterprise-level software tools are often bloated, requiring account sign-ups, synchronized bank access, or reliance on remote databases that raise privacy concerns.

***There** is a distinct necessity for a fast, light, self-contained client-side application that protects user data privacy while offering rich graphic analytics to help users immediately evaluate their spending patterns without complex configuration friction.*

3. OBJECTIVES

To engineer an intuitive, single-page application (SPA) capable of real-time balance calculations.

To implement secure local caching mechanisms to persist transaction histories across user browser sessions.

To provide graphical analytics breakdowns using category-based visualization schemas (Pie and Bar charts).

To optimize UI components to achieve extreme performance responsiveness across mobile and desktop breakpoints.

To construct a state management architecture using Context API to eliminate data drilling across deep DOM sub-trees.

4. PROJECT SCOPE

In-Scope Features: Comprehensive Create-Read-Update-Delete (CRUD) data logging ledger; analytical spending breakdowns across specified core sectors (Food, Travel, Shopping, Bills, Other); multi-mode UI (Light/Dark themes); structural local isolation via JSON strings; operational content query string filters.

Out-of-Scope / Future Enhancements: Cloud-based user authentication (Firebase/OAuth), server-side relational databases, exporting historical ledgers to Excel/CSV spreadsheets, automatic bank statement parsing API connections, and setting recurring budget threshold alerts.

5. TECHNOLOGIES USED

Core Architecture: React.js (v18+), JavaScript (ES6+), Vite Bundling Core.

Navigation Routing: React Router DOM (Single-Page App Layout Routing).

State & Cache Layers: React Context API, Web Local Storage API.

Graphic Visualization: Chart.js, React-Chartjs-2.

Interface Assets & Design: React Icons, Pure CSS Modules (utilizing systemic CSS Custom Properties/Variables)

6. APPLICATION FLOW

Entry Point: The browser hits the dashboard interface where the main root wrappers read current localized theme preferences and transaction values from localStorage.

Dashboard Display: The system renders three primary statistical tracking blocks computing real-time cash vectors, structural expense pie configurations, and an isolated lookback vector listing the five most recent transactions.

Data Ingestion: Navigating to the Transactions view allows the client to enter descriptive titles, monetary amounts, categories, and direction types (Income vs. Expense).

State Upgrades: Submitting the form writes into the global ExpenseContext, automatically prompting recalculations across all active data hooks and overwriting local cache blocks.

Analytics View: Navigating to the Analytics view charts the cumulative values into comparing graphical columns to isolate performance sectors.

Configuration Toggles: The Settings interface alters root DOM dataset attributes to toggle dark display modes or invoke data purge routines to erase localized tracking metrics.

7. KEY REACT CONCEPTS USED

Functional Components & Hooks: Utilized use State to govern micro-form elements

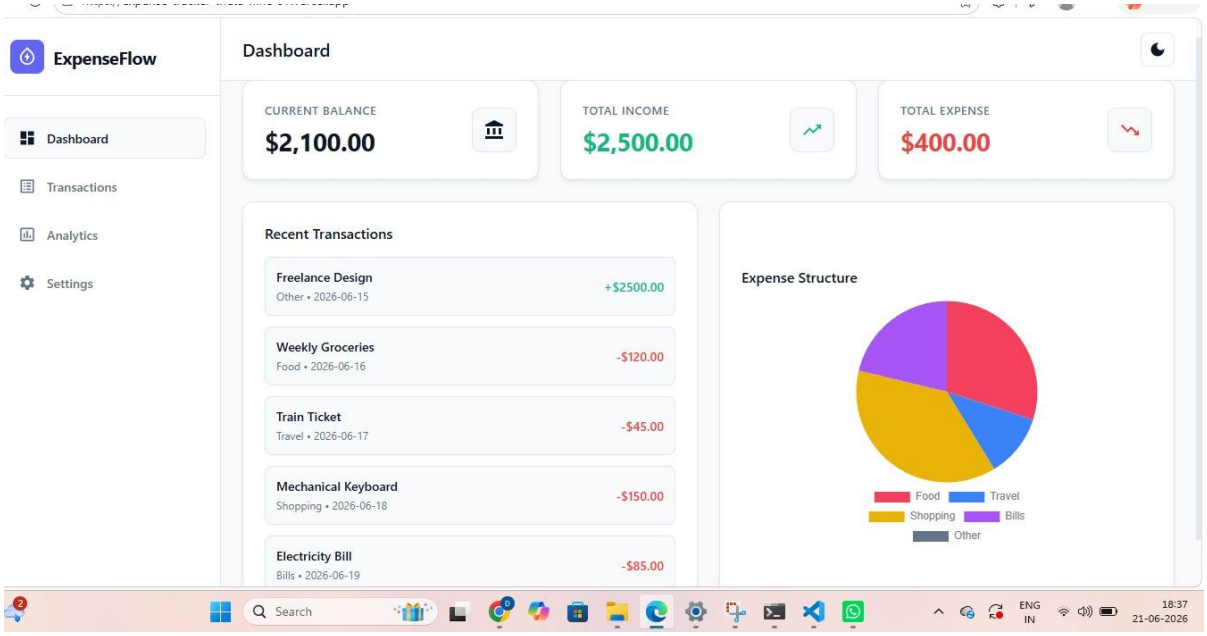
and use Effect to trigger local cache updates when global state layers change.

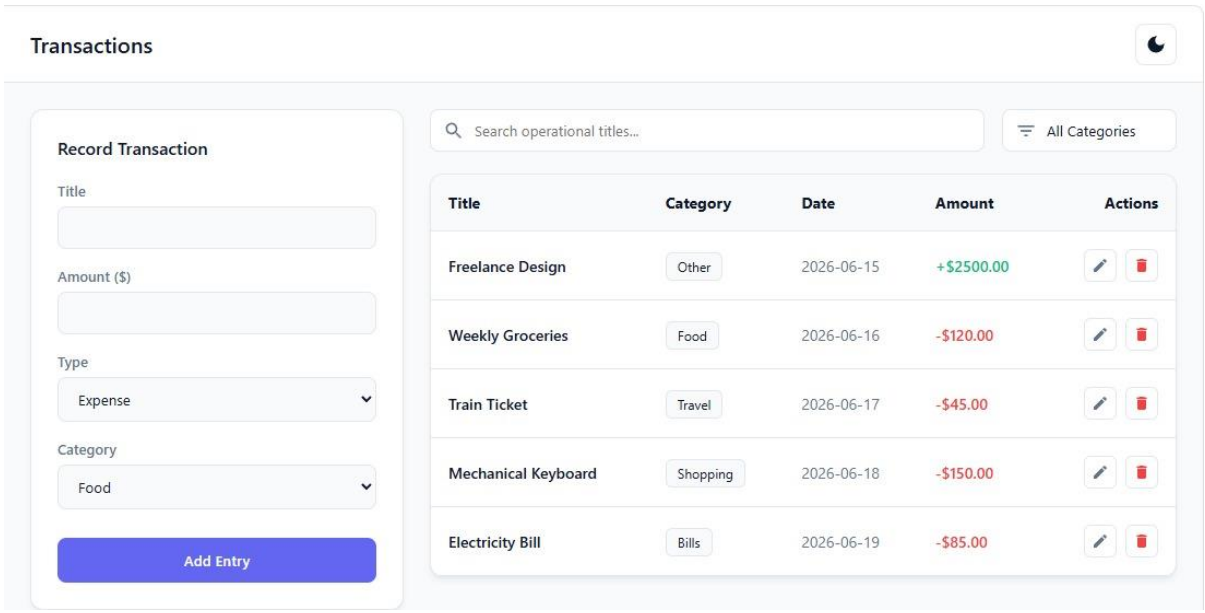
Context API (create Context & use Context): Established global stores (Expense Context and Theme Context) to share states down the tree, replacing tedious prop-drilling workflows.

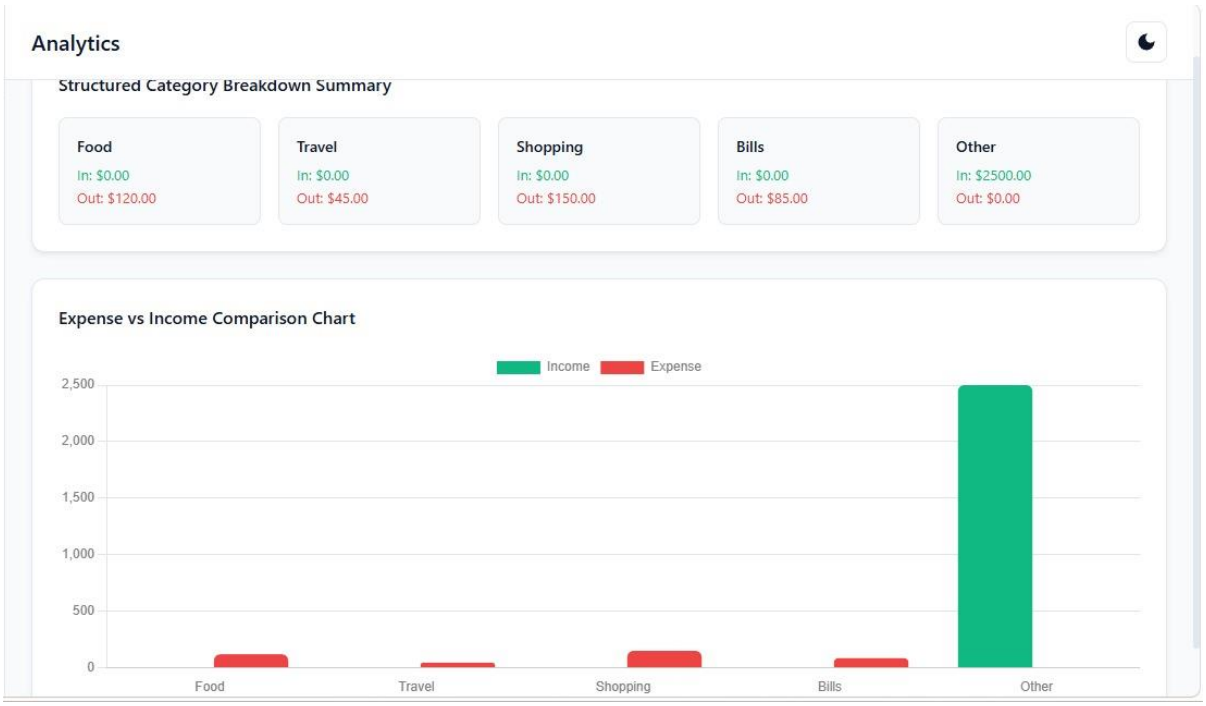
Declarative Dynamic Routing: Used Browser Router, Routes, and Route components from React Router DOM to split the user workspace cleanly without page-refresh overhead.

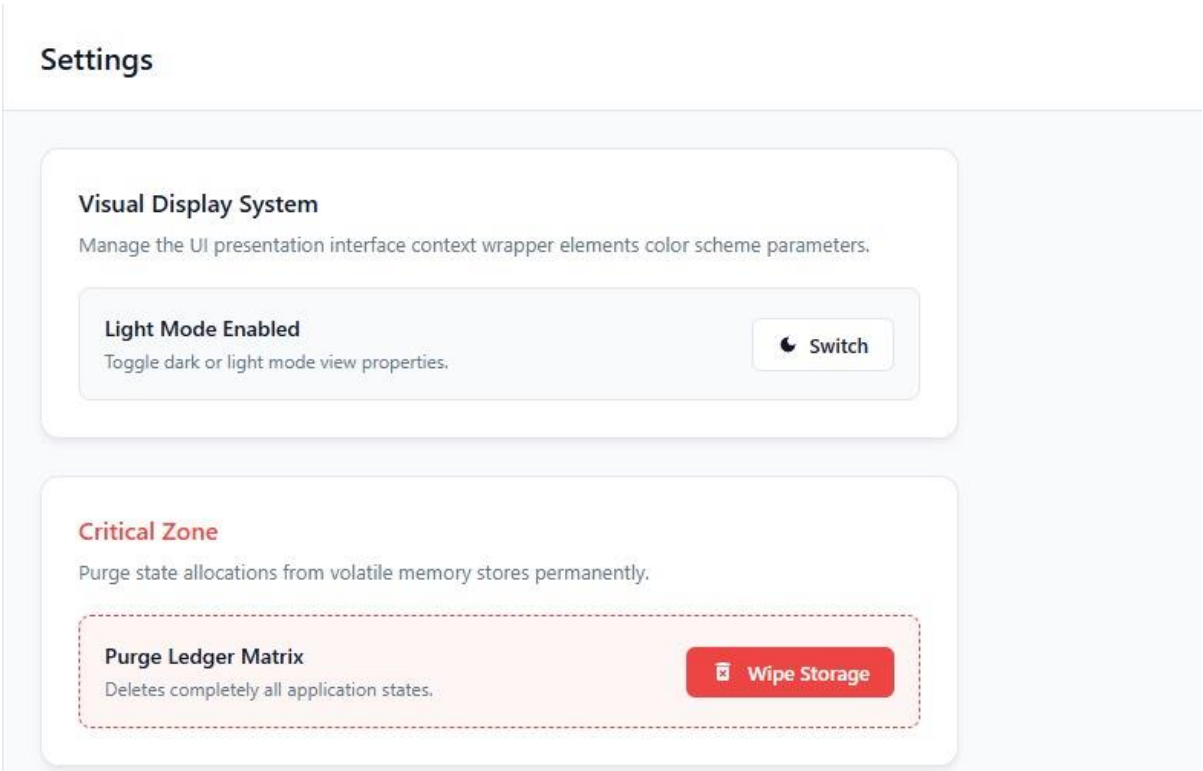
Conditional Component Rendering: Utilized structural logical statements (condition && element) to render empty fallback placeholders when the transaction arrays are empty.

8. EXPECTED OUTPUT









9. CHALLENGES FACED

Chart Refresh Lag: Updating state values sometimes caused the Chart.js instances to glitch or fail to re-render. Resolution: Solved by restructuring the calculation objects to generate brand new data references on every transaction array change, forcing the chart canvas to safely adjust its parameters.

Persistent Variable Synchronization: Toggling dark mode occasionally caused flashing

states during initial mount rendering. Resolution: Placed inline state initialization wrappers directly inside the context file to grab matching local cache properties before rendering the DOM elements.

10. LEARNING OUTCOMES

Mastered single-page application architectures using modern React patterns and Vite workflows.

Gained practical expertise managing cross-component states without external tools like Redux.

Learned how to construct a unified responsive layout utilizing custom CSS variables for light/dark themes.

Acquired experience mapping native JavaScript objects into visual canvas elements using third-party charting libraries.

11. CONCLUSION

Expense Flow successfully resolves the core challenges of modern, localized personal asset accounting by delivering an incredibly responsive UI designed without expensive server maintenance overhead.

By relying strictly on localized data arrays, it provides zero-latency interactions and bulletproof data privacy.

The completed project meets all of the required criteria for the Naan Mudhalvan Arts Internship Program 2026, acting as a production-like demonstration of a robust React-driven application architecture.